



UNIVERSIDAD NACIONAL
AUTÓNOMA DE MÉXICO

Facultad de Ingeniería



Proyecto Final: Base de Datos para un Restaurante

Materia: Bases de Datos

Profesor: Fernando Arreola Franco

Nombre: De León López Alexis

Grupo: 1

Semestre: 2026-2

Fecha de entrega: 30/05/2026

1. Introducción

En este proyecto me centro en el análisis, diseño e implementación de un sistema de base de datos relacional robusto para la gestión operativa y administrativa de un restaurante.

El problema con la administración manual o en hojas de cálculo de las comandas, el control de inventario de platillos y la generación de facturas es que propicia inconsistencias en los datos, errores de cálculo en los cobros y dificultades para auditar el rendimiento del personal. Específicamente, el seguimiento de las órdenes y el cálculo dinámico de los montos a pagar requieren una actualización en tiempo real que minimice el error humano.

Por lo que busque desarrollar una base de datos normalizada que automatice el flujo de información desde que un mesero toma una orden hasta la facturación, garantizando la integridad referencial y facilitando la extracción de métricas de negocio.

Se propone un esquema relacional implementado en PostgreSQL que hace uso intensivo de automatizaciones internas (Triggers y Funciones PL/pgSQL). Esta solución permite calcular subtotales y totales dinámicamente, validar la disponibilidad de los productos en tiempo real e indexar la información fiscal de los clientes para su rápida recuperación.

2. Plan de Trabajo

Para asegurar la correcta ejecución del proyecto, me establecí una metodología secuencial dividida en cuatro fases principales. A continuación, se describe el plan de actividades realizado.

2.1. Fases del Proyecto

1. **Fase de Análisis y Requisitos (Semana 1):** Levantamiento de requerimientos, definición de entidades principales (Empleado, Cliente, Orden, Producto) y reglas de negocio.
2. **Fase de Diseño Conceptual y Lógico (Semana 2):** Elaboración del Modelo Entidad-Relación (MER), paso al modelo relacional y normalización de las tablas hasta la Tercera Forma Normal (3FN).
3. **Fase de Implementación Física (Semana 3):** Codificación del DDL en PostgreSQL, inserción de datos de prueba (DML) y programación de Triggers y Stored Procedures.

3. Diseño

El diseño de la base de datos se llevó a cabo en dos etapas fundamentales para garantizar una estructura robusta, íntegra y sin redundancias: el Modelo Entidad-Relación y su posterior transformación al Modelo Relacional.

3.1. Modelo Entidad-Relación (MER)

En esta primera etapa conceptual, se identificaron y abstraieron los elementos clave de la operación del restaurante:

- **Entidades Fuertes:** Se definieron entidades principales con existencia independiente, tales como Producto, Orden y Factura (representando al cliente con requerimientos fiscales).
- **Subtipificación:** Se estableció una jerarquía de especialización para los empleados. A partir de una entidad genérica Empleado, se derivó el subtipo Mesero. Esto asegura que solo el personal con este rol tenga la capacidad de registrar comandas.

- **Relaciones:** Se mapearon las interacciones del negocio, destacando relaciones de uno a muchos ($1 : N$), como la de un mesero atendiendo múltiples órdenes, y relaciones de muchos a muchos ($N : M$), como la interacción principal: una orden contiene múltiples productos, y un mismo producto puede formar parte de múltiples órdenes distintas.

El diagrama fue realizado en la herramienta **draw.io**, por lo que el link para acceder al diagrama se encuentra en la carpeta del proyecto.

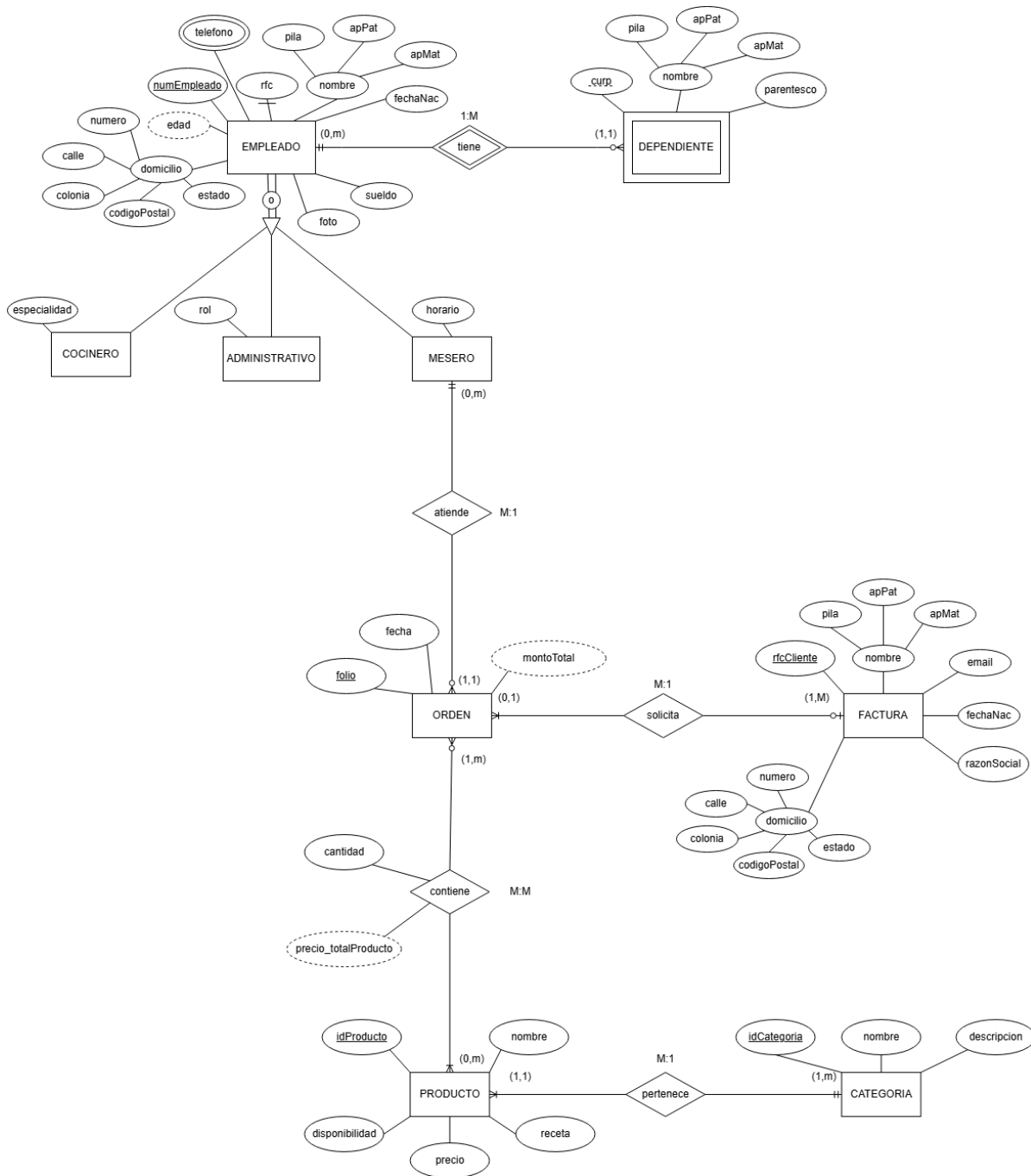


Figura 1: Modelo Entidad-Relación (MER) del sistema de restaurante.

3.2. Modelo Relacional

Una vez estructurado el MER, se procedió a realizar el mapeo formal hacia el esquema relacional, aplicando reglas de normalización (hasta la 3FN) para evitar anomalías en las operaciones de inserción, borrado y actualización:

- **Transformación de Entidades y Llaves:** Cada entidad fuerte se convirtió en una relación (tabla) independiente, dotada de una llave primaria (PRIMARY KEY). Se utilizaron secuencias y códigos alfanuméricos como identificadores (por ejemplo, folio_orden y id_producto).
- **Resolución de Relaciones $N : M$:** La relación de muchos a muchos entre las órdenes y los platillos se resolvió creando la tabla intermedia (o puente) producto_orden. Esta tabla almacena las llaves foráneas de ambas entidades y contiene atributos dependientes estrictamente de esa intersección, como la cantidad requerida y el precio_total_producto.
- **Integridad Referencial:** Se establecieron restricciones de llave foránea (FOREIGN KEY) para blindar las asociaciones, garantizando que no existan registros "huérfanos". Por ejemplo, el sistema asegura que cualquier factura emitida esté atada a una orden válida, y que toda orden pertenezca a un mesero registrado.

El diagrama fue realizado en la herramienta **dbdiagram.io**, por lo que el código para generar el diagrama se encuentra en la carpeta del proyecto.

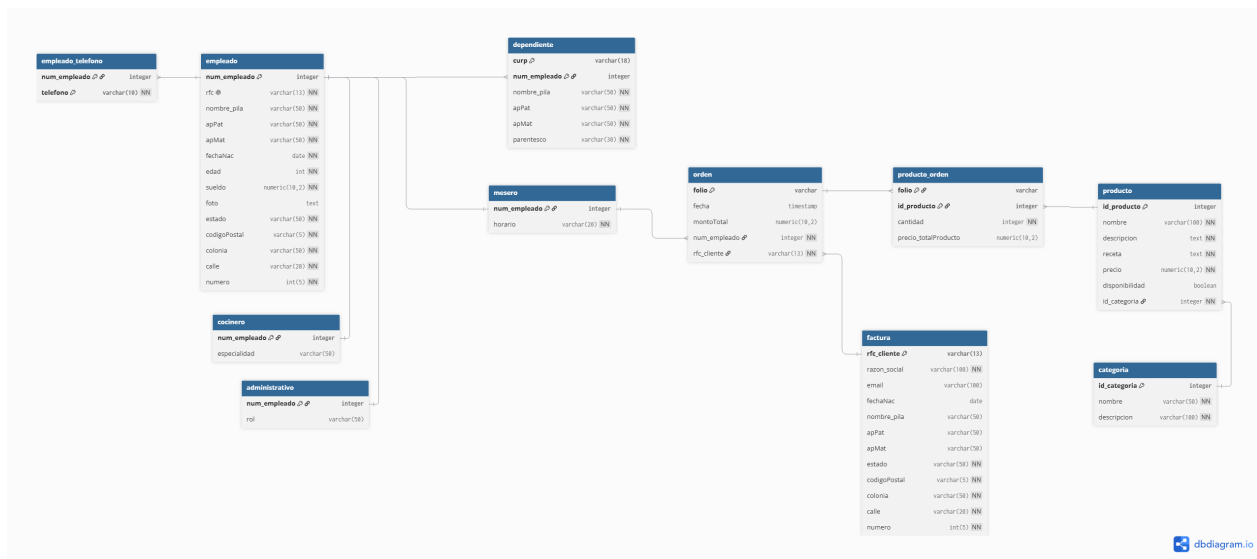


Figura 2: Modelo Relacional del sistema de restaurante.

4. Implementación

La base de datos fue materializada utilizando el sistema gestor PostgreSQL. El esquema DDL incluye la creación de tablas con restricciones PRIMARY KEY, FOREIGN KEY, y CHECK para mantener la integridad de dominio.

Mecanismos Automatizados Implementados:

1. **Secuencias Alfanuméricas:** Se implementó una secuencia (seq_folio_orden) acoplada a la función LPAD para autogenerar llaves primarias con formato comercial (ej. ORD-001), facilitando el seguimiento visual de los tickets.

```

-- Secuencia necesaria para la PK de la tabla Orden
CREATE SEQUENCE seq_folio_orden START WITH 1;

-- Secuencia para el PK de la tabla empleado
CREATE SEQUENCE IF NOT EXISTS seq_empleado_num
START WITH 1
INCREMENT BY 1;

CREATE TABLE orden (
    folio_orden TEXT DEFAULT 'ORD-' || LPAD(NEXTVAL('seq_folio_orden')::TEXT, 3, '0'),
    fecha_hora TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

```

Figura 3: Secuencia alfanumérica para generación de folios de orden.

2. **Trigger 1 - Validación y Subtotales (fn.calcular_detalle_orden):** Este disparador se ejecuta BEFORE INSERT OR UPDATE en la tabla producto_orden. Su función es consultar la tabla producto para verificar si el platillo tiene la bandera de disponibilidad en TRUE. De ser así, extrae el precio unitario, lo multiplica por la cantidad solicitada y asigna el valor a precio_total_producto. Si no está disponible, aborta la transacción lanzando una excepción.

```

-- TRIGGER 1: Valida disponibilidad y calcula el precio por producto (Cantidad * Precio)
CREATE OR REPLACE FUNCTION fn_calcular_detalle_orden()
RETURNS TRIGGER AS $$
DECLARE
    vPrecio NUMERIC(10,2);
    vDisponible BOOLEAN;
BEGIN
    -- Obtenemos el precio y disponibilidad del producto
    SELECT precio, disponibilidad INTO vPrecio, vDisponible
    FROM producto
    WHERE id_producto = NEW.id_producto;

    -- Validar disponibilidad
    IF NOT vDisponible THEN
        RAISE EXCEPTION 'El producto ID % no se encuentra disponible actualmente.', NEW.id_producto;
    END IF;

    -- Calcular el total del producto (Nombre de columna corregido)
    NEW.precio_total_producto := NEW.cantidad * vPrecio;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- CREATE TRIGGER trg_calcular_detalle_orden
-- BEFORE INSERT OR UPDATE ON producto_orden
-- FOR EACH ROW EXECUTE FUNCTION fn_calcular_detalle_orden();

```

Figura 4: Trigger de validación y cálculo de subtotales.

3. **Trigger 2 - Actualización de la Orden (fn.actualizar_total_orden):** Se ejecuta tras cualquier cambio en los detalles de la orden. Utiliza la función COALESCE y SUM para recalculer el monto_total de la tabla orden basándose en todos los productos vinculados a su folio_orden, garantizando que la cuenta del cliente sea siempre exacta.

```

-- TRIGGER 2: Actualiza el montoTotal de la Orden cada vez que se agrega/elimina un producto
CREATE OR REPLACE FUNCTION fn_actualizar_total_orden()
RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'DELETE' THEN
        -- Usamos folio_orden, monto_total y precio_total_producto
        UPDATE orden
        SET monto_total = (SELECT COALESCE(SUM(precio_total_producto), 0) FROM producto_orden WHERE folio_orden = OLD.folio_orden)
        WHERE folio_orden = OLD.folio_orden;
        RETURN OLD;
    ELSE
        -- Usamos folio_orden, monto_total y precio_total_producto
        UPDATE orden
        SET monto_total = (SELECT COALESCE(SUM(precio_total_producto), 0) FROM producto_orden WHERE folio_orden = NEW.folio_orden)
        WHERE folio_orden = NEW.folio_orden;
        RETURN NEW;
    END IF;
END;
$$ LANGUAGE plpgsql;

-- CREATE TRIGGER trg_actualizar_total_orden
-- AFTER INSERT OR UPDATE OR DELETE ON producto_orden
-- FOR EACH ROW EXECUTE FUNCTION fn_actualizar_total_orden();

```

Figura 5: Trigger de actualización del monto total de la orden.

5. Presentación

Para la capa de presentación y conexión hacia la base de datos, se optó por la **creación de Vistas Estáticas (Views) y Funciones Parametrizadas (Stored Functions) nativas de PostgreSQL**. Esta modalidad permite que cualquier aplicación cliente (ya sea un backend o una herramienta de análisis) consuma la información ya procesada, reduciendo la carga de procesamiento en el lado del servidor de aplicaciones.

Elementos de Presentación Desarrollados:

- **Simulador de Factura (vw_factura_orden):** Una vista que aplanla la complejidad relacional haciendo múltiples JOIN. Concatena domicilios y agrupa los productos consumidos en una sola cadena de texto (STRING_AGG), entregando un formato tabular listo para ser impreso o exportado a PDF.

A/Z Folio_Orden	Fecha_Emision	A/Z RFC_Cliente	Razon_Social	A/Z Atencion_A	A/Z Domicilio_Fiscal	A/Z Conceptos	123 Total_A_Pagar
ORD-001	2026-05-30 21:10:33.557	GOB1809510ABC	Juan Gómez Bernal	Juan Gómez Bernal	Galicia 144, Alamos, C.P. 03400, CDMX	Guacamole Tradicional (x1), Sopa de Tortilla (x1), Queso Fundido (x1)	365
ORD-002	2026-05-30 21:10:33.557	PERM850615DEF	María Pérez Muñoz	María Pérez Muñoz	Hidalgo 22, Santa Cruz, C.P. 54050, EdoMex	Rib Eye 400g (x1), Limonada Mineral (x2), Pastel de Chocolate (x1)	625
ORD-003	2026-05-30 21:10:33.557	LOP5901220GHI	Sergio López Soto	Sergio López Soto	Rio Lerma 90, Cuauhtémoc, C.P. 06500, CDMX	Rib Eye 400g (x2), Guacamole Tradicional (x1), Ensalada César (x1)	1,150
ORD-004	2026-05-30 21:10:33.557	XAOX010101000	Público en General	Público General NA	NA SN, NA, C.P. 00000, CDMX	Tacos al Pastor (Orden) (x3)	330
ORD-005	2026-05-30 21:10:33.557	RAMA176030KIL	Ana Ramírez Arce	Ana Ramírez Arce	Picacho 88, Jardines, C.P. 14200, CDMX	Salmon a las Finas Hierbas (x1), Ensalada César (x1)	490
ORD-006	2026-05-30 21:10:33.557	DIAH408911MNO	Hugo Díaz Hernández	Hugo Díaz Hernández	Sur 20 41, La Peña, C.P. 57200, EdoMex	Pechuga a la Cordon Bleu (x2), Queso Fundido (x2)	740
ORD-007	2026-05-30 21:10:33.557	XAOX010101000	Público en General	Público General NA	NA SN, NA, C.P. 00000, CDMX	Ensalada César (x2)	280
ORD-008	2026-05-30 21:10:33.557	CRUR881116STU	René Cruz Ramos	René Cruz Ramos	Amores 55, Del Valle, C.P. 03100, CDMX	Sopa de Tortilla (x1), Pechuga a la Cordon Bleu (x2)	535
ORD-009	2026-05-30 21:10:33.557	GUTL930708VWX	Laura Gutiérrez Luna	Laura Gutiérrez Luna	Colina 12, Lomas Verdes, C.P. 53100, EdoMex	Pastel de Chocolate (x1), Limonada Mineral (x1)	130
ORD-010	2026-05-30 21:10:33.557	HERF990119YZA	Fernando Hernández Flores	Fernando Hernández Flores	Mexicali 6, Condesa, C.P. 06700, CDMX	Rib Eye 400g (x1), Salmon a las Finas Hierbas (x1), Guacamole Tradicional (x1)	890

Figura 6: Vista para simulación de factura.

- **Métricas de Rendimiento (fn_ventas_por_periodo & fn_rendimiento_mesero_hoy):** Funciones que reciben parámetros de entrada (fechas o ID de empleados) y devuelven tablas resumidas con la cantidad de órdenes atendidas y los montos acumulados, funcionando como un tablero de control gerencial.

123 total_ventas	123 monto_acumulado
10	5,475

Figura 7: Funciones para métricas de rendimiento.

6. Conclusiones

El desarrollo de este proyecto representó un desafío que me permitió aplicar gran parte de los conocimientos adquiridos a lo largo del curso, los cuales me preparan para enfrentar escenarios reales en la gestión de bases de datos empresariales.

Siendo que uno de los principales retos técnicos durante la etapa de implementación fue la depuración de la lógica interna de PostgreSQL con respecto a la sensibilidad de mayúsculas y minúsculas (camelCase vs. snake_case). Se presentaron errores de sintaxis donde los registros NEW de los triggers no reconocían columnas como `precio_total_producto` debido a inconsistencias de nomenclatura, lo que demandó un proceso riguroso de estandarización en todo el código fuente.

Un gran acierto del diseño fue delegar la lógica matemática y las validaciones de negocio directamente al motor de la base de datos mediante Triggers. Esto asegura que, independientemente de la interfaz gráfica o aplicación que inserte los datos en el futuro, es imposible registrar un producto agotado o cobrar una orden con cálculos erróneos.

Este proyecto consolidó mi comprensión sobre la importancia de una correcta normalización y el uso eficiente de secuencias y restricciones. El resultado es un esquema escalable, robusto y preparado para ser el núcleo de un sistema de información de nivel empresarial. Además de ayudarme a practicar la identificación de requerimientos según el contexto del negocio, lo que es fundamental para cualquier desarrollador de bases de datos.